Setup, Integration & Run Guide

Step-by-step: install, seed, connect, send e-mail, fill Google Sheet, go live

Version 1.0 · October 2026

Laravel 12 · React · MySQL · GSAP · Sneat admin

Contents

Contents 2

1. What you received 3

How the pieces connect . . . 3

2. Quick start (10 minutes) 4

3. Backend in detail 5

3.1 Requirements and PHP settings . . . 5

3.2 The .env file explained . . . 5

3.3 Public API (what the React site calls) . . . 5

3.4 Artisan commands you will use . . . 6

4. Using the admin dashboard 7

4.1 Live search (top bar) . . . 7

4.2 Writing a long blog post with images . . . 7

4.3 Adding a project with lots of information . . . 8

4.4 Theme colours and site look . . . 8

5. E-mail: contact messages arrive in your inbox 9

Local testing (no real e-mail) . . . 9

Gmail (free) . . . 9

6. Google Sheet + Excel records 10

6.1 Connect a Google Sheet (about 5 minutes) . . . 10

6.2 Excel . . . 10

7. Connecting the React frontend 11

Build for production . . . 11

8. Building the Laravel project yourself (file by file) 12

9. Going live (production) 14

9.1 Recommended layout . . . 14

9.2 Ubuntu VPS (Nginx + PHP-FPM + MySQL) . . . 14

9.3 cPanel / shared hosting . . . 14

9.4 Security checklist . . . 15

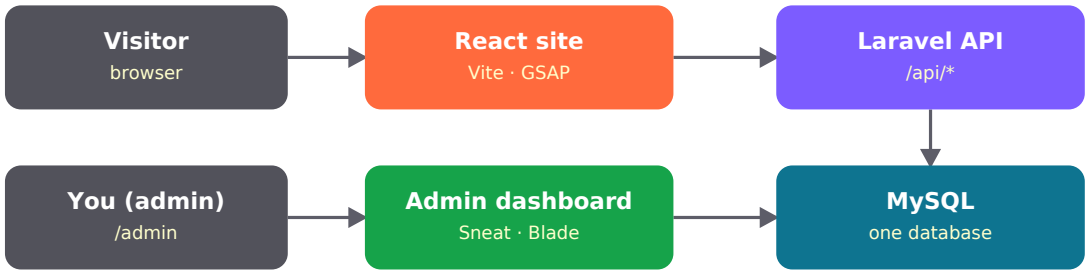
10. Troubleshooting 16

1. What you received

A complete full-stack portfolio: a React website, a Laravel REST API, a Sneat-based admin dashboard and demo data — all talking to one MySQL database.

Folder	What it is	Runs on
`backend/`	Laravel 12 project: REST API (`/api/...`), admin dashboard (`/admin`), migrations, seeders, mail, Google Sheet + Excel export, Sneat assets, TinyMCE editor	http://localhost:8000
`frontend/`	React 18 + Vite site: GSAP / ScrollTrigger / Lenis animations, 3D rings, live search, dark/light mode, colour settings	http://localhost:5173
`docs/`	These PDFs, the Google Apps Script (`google-apps-script.gs`) and a sample Nginx config	—

How the pieces connect



- **Visitors** open the React site. It reads everything (hero text, projects, posts, colours…) from `GET /api/...`.
- **You** log in to `/admin`, edit content, upload images/videos, read contact messages. Both sides use the **same MySQL database**.
- A contact form submission is saved in MySQL, **emailed to you**, **pushed to your Google Sheet**, and can be **downloaded as Excel** any time.

2. Quick start (10 minutes)

You need: **PHP 8.2+**, **Composer 2**, **MySQL 8 / MariaDB 10.6+**, **Node.js 18+**. On Windows the easiest stack is **Laragon** (PHP + MySQL + Composer + Node in one installer). On Linux/Mac install them with your package manager.

Step 1 — create the database

```
$ mysql -u root -p
mysql> CREATE DATABASE portfolio CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
mysql> EXIT;
# (phpMyAdmin: New → name "portfolio" → collation utf8mb4_unicode_ci → Create)
```

Step 2 — backend

```
$ cd backend
$ composer install
$ cp .env.example .env          # Windows: copy .env.example .env
# open .env and set:  DB_DATABASE=portfolio  DB_USERNAME=root  DB_PASSWORD=your-password
$ php artisan portfolio:install   # key + tables + demo data + storage link
$ php artisan serve               # http://localhost:8000
```

`portfolio:install` runs `key:generate`, `migrate`, `db:seed` and `storage:link` for you. Use `php artisan portfolio:install --fresh` to wipe everything and start again (this **deletes all data**).

Step 3 — frontend

```
$ cd frontend
$ npm install
$ cp .env.example .env          # VITE_API_URL=http://localhost:8000
$ npm run dev                    # http://localhost:5173
```

Step 4 — open it

URL	What
http://localhost:5173	The public website (React)
http://localhost:8000/admin	Admin dashboard — login `admin@example.com` / `password123`
http://localhost:8000/api/bootstrap	Raw JSON the website reads (good for debugging)

> **Change the admin password now**
>
> Admin → top-right avatar → My profile. Also set `ADMIN_EMAIL` / `ADMIN_PASSWORD` in `.env` **before** seeding on a live server.

> **Banglay shortcut**
>
> Database banao → `backend` e `composer install` → `.env` e DB info → `php artisan portfolio:install` → `php artisan serve`. Arekta terminal e `frontend` e `npm install` → `npm run dev`. Site: 5173, Admin: 8000/admin.

3. Backend in detail

3.1 Requirements and PHP settings

Item	Needed	Notes
PHP	8.2 or newer	Extensions: ctype, curl, dom, fileinfo, mbstring, openssl, pdo_mysql, tokenizer, xml, **zip** (zip = Excel export; without it CSV is offered)
`upload_max_filesize`	≥ 64M (200M for videos)	php.ini; also `post_max_size` slightly larger, `max_execution_time=300`
Composer	2.x	`composer install` downloads Laravel into `vendor/`
MySQL / MariaDB	8.0 / 10.6+	Charset utf8mb4

3.2 The .env file explained

Key	Meaning
`APP_URL`	Where Laravel runs (used for image URLs and e-mail links)
`APP_TIMEZONE`	Dates in admin / Excel (default Asia/Dhaka)
`DB_*`	MySQL host, database, user, password
`FRONTEND_URL`	React site origin(s) allowed by CORS — comma separated: `http://localhost:5173,https://yourdomain.com`
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	Account created by the seeder
`MAIL_*`	E-mail sending (section 5)
`SESSION_DRIVER`, `CACHE_STORE`, `QUEUE_CONNECTION`	file / file / sync — no extra tables or workers needed

> **After editing .env**
>
> Run `php artisan config:clear` so Laravel re-reads it.

3.3 Public API (what the React site calls)

Method & URL	Returns
`GET /api/bootstrap`	settings, colours, services, skills, timeline, FAQs, categories (loaded once at start)
`GET /api/projects` · `/api/projects/{slug}`	list · full case study (gallery, video, rich-text HTML, next project)
`GET /api/posts` · `/api/posts/{slug}`	list · full article HTML + 3 related posts (also counts a view)

Method & URL	Returns
`GET /api/search?q=laravel`	live-search suggestions: type, title, excerpt, url, image
`POST /api/contact`	name, email, message, project_type, budget → saves + e-mail + Google Sheet (max 5/min per IP)
`POST /api/subscribe`	newsletter e-mail
`GET /api/image/{path}`	CORS-enabled image (used by the pixel hero portrait)

```
// POST /api/contact  →  { "ok": true, "message": "Thanks! Your message has been sent." }
// Validation error → HTTP 422 { "message": "...", "errors": { "email": ["The email field must be a valid email
      address."] } }
```

3.4 Artisan commands you will use

Command	Does
`php artisan portfolio:install [--fresh]`	full first-time setup
`php artisan migrate`	apply new migrations only
`php artisan db:seed`	insert demo content again (safe: uses updateOrCreate)
`php artisan db:seed --class=AdminUserSeeder`	only (re)create the admin user
`php artisan storage:link`	public/storage → storage/app/public (images/videos URLs)
`php artisan config:clear` / `cache:clear`	after editing .env / settings
`php artisan route:list`	see every URL

4. Using the admin dashboard

Open http://localhost:8000/admin. Everything you change appears on the website immediately — no code, no rebuild.

Menu	What you manage
Dashboard	counters, 6-month message chart, most-read posts, recent messages, setup checklist
Projects	unlimited projects: cover, demo video (upload or YouTube/Vimeo URL), gallery, tags, links, 3 rich-text sections + a huge "Full case study" editor, SEO
Blog posts	rich article editor with images anywhere (start / middle / end), left / right / centre alignment, captions, code, tables, video
Categories · Skills · Services · Journey timeline · FAQs	content blocks of the About / Projects / Blogs / Contact pages
Messages	inbox, status (new / read / replied), notes, **Download Excel**, CSV, **Sync to Google Sheet**
Subscribers	newsletter list + CSV export
Theme colours	add / edit the accent colours visitors can pick; mark one as default
Media library	every uploaded image/video; copy URL; delete
Site settings	hero text & photo, stats, About text, contact e-mail, Google Sheet URL, social links, default theme / font

4.1 Live search (top bar)

Click the search box (or press /) and type. After a short pause the dashboard queries the database and shows suggestions with a type badge, a short description, a link and a **thumbnail on the right**. Use ↑ ↓ and Enter. It searches projects, posts, messages, categories, skills, services, FAQs and admin pages.

4.2 Writing a long blog post with images

1. Blog posts → **New post**. Fill title, category, excerpt, **cover image** (shown on cards and at the top of the article).
2. In the **Content** editor type your text. Use the *Blocks* drop-down for Heading 2 / Heading 3 — Heading 2 becomes the table-of-contents on the site.
3. **Image at the start:** put the cursor on the first empty line → Insert → Image → *Upload* (or the folder icon to pick from the media library).
4. **Align it:** click the image → toolbar buttons *Align left / centre / right*. Left & right make text wrap around the image; centre puts it on its own line. In the image dialog → Class you can choose *Full width / Medium / Small*.
5. **Image in the middle / end:** move the cursor to that paragraph and repeat. Tick *Show caption* in the image dialog for a caption under the picture.

6. Code: Insert → Code sample. Video: Insert → Media → paste a YouTube / Vimeo / .mp4 link. Tables: Table menu. Full-screen writing: the arrows icon.
7. Choose **Published**, set the publish date (future dates stay hidden until then) and **Save**. Open the site → Blogs to see it.

> **Image size tip**
>
> Resize photos to ~1600 px wide (under 500 KB) before uploading. Uploads up to 12 MB are accepted by the editor, 8 MB for cover images.

4.3 Adding a project with lots of information

Projects → **New project**. The form is split into cards: Basics → Media → Links & tech → Story (Overview, Challenge, Solution, Results, Key features) → **Full case study** (an unlimited rich-text editor — add headings, images, tables, code) → SEO. Upload several screenshots at once in *Gallery*; untick **Keep** on an existing image to remove it. Toggle **Featured** to show it on the home page.

4.4 Theme colours and site look

Theme colours → **New colour** (name + colour picker). Tick *Default accent* for the colour new visitors see first. All colours appear as swatches in the website's ⚙ Settings drawer, where each visitor can pick one (and also light/dark, font, motion on/off). Site settings → *Appearance* sets the default theme and font.

5. E-mail: contact messages arrive in your inbox

When someone submits the contact form Laravel (1) saves the message, (2) e-mails **every address in Site settings → Contact & e-mail → "Receive messages at"**, (3) pushes a row to Google Sheet, (4) optionally sends an auto-reply to the visitor. If e-mail fails the message is **still saved** (check Admin → Messages: the *Mail* column).

Local testing (no real e-mail)

Keep `MAIL_MAILER=log`. Submit the form, then open `backend/storage/logs/laravel.log` — the full e-mail text is there.

Gmail (free)

1. Google Account → Security → turn on **2-Step Verification**.
2. Security → **App passwords** → create one named "portfolio" → copy the 16 letters.
3. Edit `.env`:
4. Run `php artisan config:clear`, then send a test message from the website.

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=youraddress@gmail.com
MAIL_PASSWORD=abcdefghijklmnop      # the app password, not your Gmail password
MAIL_FROM_ADDRESS=youraddress@gmail.com
MAIL_FROM_NAME="${APP_NAME}"
```

Other providers: **Mailtrap** (testing inbox: host `sandbox.smtp.mailtrap.io`, port 2525), **Brevo / SendGrid / Mailgun / your cPanel mail** (use the SMTP details they give you; for port 465 add `MAIL_SCHEME=smtps`).

6. Google Sheet + Excel records

6.1 Connect a Google Sheet (about 5 minutes)

1. Create a new Google Sheet (e.g. "Portfolio messages").
2. Extensions → **Apps Script**. Delete the sample code and paste the contents of `docs/google-apps-script.gs`.
3. Change `SECRET` to any long random text (remember it).
4. Click **Deploy → New deployment → ⚙ Web app**. *Execute as:* **Me**. *Who has access:* **Anyone**. Deploy and **authorise**.
5. Copy the **Web app URL** (ends with `/exec`).
6. Admin → Site settings → *Contact & e-mail*: paste the URL in **Google Sheet webhook URL** and the same secret in **Google Sheet secret token** → Save.
7. Send a test message from the website — a new row appears. Messages sent earlier: Admin → Messages → **Sync to Google Sheet**.

> **If you edit the Apps Script later**
>
> Deploy → Manage deployments → ✏ → Version: **New version** → Deploy. The URL stays the same.

6.2 Excel

Admin → Messages → **Download Excel** gives `contact-messages-YYYY-MM-DD.xlsx` (ID, date, name, email, project type, budget, message, status, e-mail sent, in Google Sheet, note). CSV is also offered. Because the Google Sheet is always up to date you can also use File → Download → Microsoft Excel (.xlsx) there.

7. Connecting the React frontend

- `frontend/.env` → `VITE_API_URL=http://localhost:8000` (production: `https://api.yourdomain.com`). Restart `npm run dev` after changing it.
- Backend `.env` → `FRONTEND_URL` must contain the exact origin of the React site (scheme + host + port) or the browser blocks requests (CORS error).
- All network code lives in `frontend/src/api.js`. Pages fetch with the `useApi()` hook; global data (settings, colours…) comes from `SiteProvider` (`useSite()`).
- If the site shows **"Backend not reachable"**: Laravel is not running, the URL is wrong or `FRONTEND_URL` is missing.

Build for production

```
$ cd frontend
$ npm run build        # creates frontend/dist  (static files)
$ npm run preview      # test the production build locally
```

Upload the **contents of** `dist/` to any static host (Netlify, Vercel, Cloudflare Pages, cPanel `public_html`, Nginx). The site uses real URLs (`/projects/my-project`), so the host must serve `index.html` for unknown paths:

Host	Setting
Netlify	file `public/_redirects` is included: `/* /index.html 200`
Apache / cPanel	file `public/.htaccess` is included (copied into `dist/`)
Nginx	`location / { try_files $uri /index.html; }`
Vercel	file `vercel.json` is included (rewrite everything to `/index.html`)

8. Building the Laravel project yourself (file by file)

The zip already contains the finished project. This chapter shows how every file was created, so you can rebuild or extend it. Run commands from the folder where you want `backend/`.

```
$ composer create-project laravel/laravel backend "^12.0"
$ cd backend
# --- models + migration ---
$ php artisan make:model Setting
$ php artisan make:model Media
$ php artisan make:model ThemeColor
$ php artisan make:model Category
$ php artisan make:model Project
$ php artisan make:model Post
$ php artisan make:model Skill
$ php artisan make:model Service
$ php artisan make:model TimelineItem
$ php artisan make:model Faq
$ php artisan make:model ContactMessage
$ php artisan make:model Subscriber
$ php artisan make:migration create_portfolio_tables
# --- controllers / middleware / mail / seeders / command ---
$ php artisan make:controller Api/PublicController
$ php artisan make:controller Admin/CrudController
$ php artisan make:controller Admin/ProjectController       # + Post, Category, Skill, Service, Timeline, Faq,
      Color
$ php artisan make:controller Admin/{Auth,Dashboard,Message,Media,Settings,Search,Profile,Subscriber}Controller
$ php artisan make:middleware AdminOnly
$ php artisan make:mail ContactReceived
$ php artisan make:mail ContactAutoReply
$ php artisan make:seeder PortfolioSeeder
$ php artisan make:seeder AdminUserSeeder
$ php artisan make:command PortfolioInstall
# --- then paste the code from the zip into those files ---
```

File (backend/…)	Purpose
`bootstrap/app.php`	registers `routes/api.php` and the `admin` middleware alias
`routes/api.php`, `routes/web.php`	public API · admin routes
`config/site.php`	schema of every editable site setting (add a field here → it appears in admin + API)
`config/cors.php`	allows the React origin (`FRONTEND_URL`)
`database/migrations/…create_portfolio_tables.php`	all tables (settings, theme_colors, media, categories, projects, posts, skills, services, timeline_items, faqs, contact_messages, subscribers + users.is_admin)
`database/seeders/*`, `seeders/assets/*`	demo content + generated images/videos (`tools/generate_seed_assets.py` recreates them)
`app/Models/*`	Eloquent models; `Project`/`Post::toApi()` shape the JSON
`app/Support/Uploader.php`	stores uploads + registers them in the media library
`app/Support/XlsxWriter.php`	dependency-free Excel (.xlsx) writer

File (backend/…)	Purpose
`app/Support/GoogleSheet.php`	posts a message to the Apps Script webhook
`app/Support/SearchService.php`	public + admin live-search queries
`app/Http/Controllers/Api/PublicController.php`	all public endpoints incl. contact pipeline
`app/Http/Controllers/Admin/CrudController.php`	the generic create/edit/delete engine (field types: text, editor, image, gallery…)
`…/Admin/ProjectController.php` etc.	only describe fields + columns (≈40 lines each)
`app/Mail/*`, `resources/views/mail/*`	the two e-mails
`resources/views/admin/*`	Sneat layout, menu, navbar, CRUD views, dashboard, settings, media, messages
`public/sneat/assets`	your Sneat template CSS/JS/fonts
`public/vendor/tinymce`	self-hosted rich-text editor
`public/admin-assets/admin.{css,js}`	our animations, live search, editor setup
`app/Console/Commands/PortfolioInstall.php`	`php artisan portfolio:install`

> **Why `admin-assets` and not `public/admin`?**
>
> A real folder called `public/admin` would be served by the web server instead of the `/admin` route. Never name a public folder like a route.

9. Going live (production)

9.1 Recommended layout

Domain	Serves	How
`yourdomain.com`	React site	static files from `frontend/dist`
`api.yourdomain.com`	Laravel API + `/admin`	Nginx/Apache → `backend/public`

9.2 Ubuntu VPS (Nginx + PHP-FPM + MySQL)

```
$ sudo apt install nginx mysql-server php8.3-fpm php8.3-{mysql,mbstring,xml,curl,zip,bcmath,intl} composer unzip
$ sudo mysql -e "CREATE DATABASE portfolio CHARACTER SET utf8mb4; CREATE USER 'pf'@'localhost' IDENTIFIED BY
      'STRONG_PASSWORD'; GRANT ALL ON portfolio.* TO 'pf'@'localhost';"
# upload backend/ to /var/www/portfolio-api
$ cd /var/www/portfolio-api && composer install --no-dev --optimize-autoloader
$ cp .env.example .env && nano .env      # APP_ENV=production APP_DEBUG=false APP_URL=https://api.yourdomain.com
                                         # FRONTEND_URL=https://yourdomain.com  DB_*  ADMIN_*  MAIL_*
$ php artisan key:generate
$ php artisan migrate --force
$ php artisan db:seed --class=AdminUserSeeder   # only the admin; add your own content in /admin
$ php artisan storage:link
$ sudo chown -R www-data:www-data storage bootstrap/cache
$ php artisan config:cache && php artisan route:cache && php artisan view:cache
```

```
# /etc/nginx/sites-available/portfolio-api
server {
  server_name api.yourdomain.com;
  root /var/www/portfolio-api/public;
  index index.php;
  client_max_body_size 210M;                # video uploads
  location / { try_files $uri $uri/ /index.php?$query_string; }
  location ~ \.php$ { include snippets/fastcgi-php.conf; fastcgi_pass unix:/run/php/php8.3-fpm.sock; }
  location ~ /\.(?!well-known) { deny all; }
}
$ sudo certbot --nginx -d api.yourdomain.com -d yourdomain.com     # free HTTPS
```

Also raise `upload_max_filesize` / `post_max_size` in `/etc/php/8.3/fpm/php.ini` and restart PHP-FPM. The React `dist/` folder goes in its own server block with `try_files $uri /index.html;`.

9.3 cPanel / shared hosting

1. Create a subdomain `api` and set its **document root to** `backend/public` (keep the rest of `backend/` outside `public_html`).
2. Create the MySQL database + user in cPanel, put the details in `.env`.
3. Run `composer install --no-dev` (Terminal/SSH) or upload `vendor/` built on your PC with the same PHP version.
4. Run `php artisan migrate --force`, the admin seeder, and `php artisan storage:link` (or create the symlink `public/storage → ../storage/app/public` in File Manager / SSH).
5. Upload `frontend/dist` contents to `public_html`.

9.4 Security checklist

- `APP_DEBUG=false`, `APP_ENV=production`, HTTPS everywhere.
- Changed admin e-mail/password; delete demo content you do not need.
- `FRONTEND_URL` lists only your real domains.
- Backups: `mysqldump -u pf -p portfolio > backup.sql` and `tar czf uploads.tgz storage/app/public` (daily cron).
- Keep packages updated: `composer update` / `npm update`, test, deploy.
- The contact endpoint is limited to 5 requests/minute per IP and has a hidden honeypot field against bots.

10. Troubleshooting

Problem	Fix
Site says "Backend not reachable"	Start `php artisan serve`; check `VITE_API_URL`; open `/api/bootstrap` in the browser
Browser console: CORS error	Add the site origin to `FRONTEND_URL` in backend `.env` → `php artisan config:clear`
Images / videos are broken	Run `php artisan storage:link`; check `APP_URL` matches the URL you open
Upload fails / "post too large"	Raise `upload_max_filesize`, `post_max_size` (php.ini) and Nginx `client_max_body_size`; restart PHP
Admin shows 419 "Page expired"	Reload the page (CSRF token expired) or clear cookies
500 error	Open `storage/logs/laravel.log` — the last lines name the problem. Check `.env` DB details
"could not find driver"	Enable `pdo_mysql` in php.ini
Excel button gives CSV	Enable the PHP `zip` extension
No e-mail received	See Admin → Messages → Mail column; `MAIL_MAILER` must be `smtp`; check spam; read `laravel.log` for the SMTP error
Google Sheet column stays "no"	Web app access must be **Anyone**; secret must match; URL must end with `/exec`; use the Sync button
Hero portrait looks like a dummy face	Admin → Site settings → Home hero → upload your photo (dark/plain background works best)
Search shows nothing	Type at least 2 letters; only *published* items appear

Banglay

Kono error hole prothome `backend/storage/logs/laravel.log` er shesh line dekho — shetai asol karon bole dey. 90% somoy `.env` (DB/URL) ba `storage:link` bad jaoya.

Licences

Sneat is a commercial/MIT-style template — keep your own licence for it. TinyMCE is bundled in its free GPL edition (`license_key: gpl`). GSAP is free for standard use; Laravel, React, Vite, Lenis are MIT.